Last code AutoEncoder

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt


from sklearn.preprocessing import MinMaxScaler

from sklearn.metrics import mean_squared_error

from sklearn.metrics import f1_score, precision_score, recall_score


from tensorflow.keras.models import Model

from tensorflow.keras.layers import Input, Dense



df1 = pd.read_excel("test3 2023.xlsx")

df2 = pd.read_excel("test3 2024.xlsx")

df3 = pd.read_excel("test3 2025.xlsx")


df = pd.concat([df1, df2, df3], ignore_index=True)


df['일시'] = pd.to_datetime(df['일시'])



df['날짜'] = df['일시'].dt.date
```

```
daily_df = df.groupby('날짜')['누적강수량(mm)'].max().reset_index()

data = daily_df[['누적강수량(mm)']]



scaler = MinMaxScaler()
scaled_data = scaler.fit_transform(data)



n_past = 7

X = []
y = []

for i in range(n_past, len(scaled_data)):
    X.append(scaled_data[i-n_past:i].flatten())
    y.append(scaled_data[i,0])

X = np.array(X)
y = np.array(y)



input_dim = X.shape[1]
```

```
input_layer = Input(shape=(input_dim,))

encoded = Dense(32, activation='relu')(input_layer)
encoded = Dense(16, activation='relu')(encoded)
encoded = Dense(8, activation='relu')(encoded)

output = Dense(1, activation='linear')(encoded)

model = Model(inputs=input_layer, outputs=output)

model.compile(optimizer='adam', loss='mse')



history = model.fit(
    X,
    y,
    epochs=100,
    batch_size=16,
    validation_split=0.2,
    verbose=1
)
```

```
pred = model.predict(X)



real = scaler.inverse_transform(y.reshape(-1,1))

pred = scaler.inverse_transform(pred)



mse = mean_squared_error(real, pred)

rmse = np.sqrt(mse)



print("MSE :", round(mse,3))

print("RMSE :", round(rmse,3))



threshold = 50   # mm



real_binary = (real >= threshold).astype(int)

pred_binary = (pred >= threshold).astype(int)



precision = precision_score(real_binary, pred_binary)

recall = recall_score(real_binary, pred_binary)

f1 = f1_score(real_binary, pred_binary)
```

```
print("Precision:", round(precision,3))
print("Recall:", round(recall,3))
print("F1-score:", round(f1,3))



dates = daily_df['날짜'][n_past:].astype(str)

plt.figure(figsize=(18,6))

plt.plot(dates, real, label='Real Rainfall', color='tab:blue')
plt.plot(dates, pred, label='AutoEncoder Prediction', color='tab:orange')

step = 15

plt.xticks(
    np.arange(0, len(dates), step),
    dates[::step],
    rotation=45
)

plt.xlabel("Date")
plt.ylabel("Rainfall (mm)")
plt.title("AutoEncoder Rainfall Prediction")
```

```
plt.legend()
plt.grid(alpha=0.3)


plt.tight_layout()
plt.show()
```

Last code LSTM

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error
from sklearn.metrics import f1_score, precision_score, recall_score

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense



df1 = pd.read_excel("test3 2023.xlsx")
df2 = pd.read_excel("test3 2024.xlsx")
df3 = pd.read_excel("test3 2025.xlsx")

df = pd.concat([df1, df2, df3], ignore_index=True)



df = df.rename(columns={
    '누적강수량(mm)': 'rainfall',
    '일시': 'datetime'
})
```

```
df['datetime'] = pd.to_datetime(df['datetime'])


df = df.fillna(df.select_dtypes(include=np.number).mean())


df['date'] = df['datetime'].dt.date

daily_df = df.groupby('date')['rainfall'].max().reset_index()


data = daily_df[['rainfall']]

scaler = MinMaxScaler()
scaled_data = scaler.fit_transform(data)


X = []
y = []
n_past = 7

for i in range(n_past, len(scaled_data)):
    X.append(scaled_data[i-n_past:i, 0])
    y.append(scaled_data[i, 0])
```

```
X = np.array(X)
y = np.array(y)

X = X.reshape(X.shape[0], X.shape[1], 1)

model = Sequential()
model.add(LSTM(64, input_shape=(X.shape[1], 1)))
model.add(Dense(1))

model.compile(optimizer='adam', loss='mse')

model.fit(X, y, epochs=100, batch_size=16, verbose=1)

predictions = model.predict(X)

real = scaler.inverse_transform(y.reshape(-1,1))
pred = scaler.inverse_transform(predictions)

mse = mean_squared_error(real, pred)
```

```
print("LSTM MSE:", mse)


threshold = 50   # mm


real_binary = (real >= threshold).astype(int)
pred_binary = (pred >= threshold).astype(int)


precision = precision_score(real_binary, pred_binary)
recall = recall_score(real_binary, pred_binary)
f1 = f1_score(real_binary, pred_binary)

print("Precision:", round(precision, 3))
print("Recall:", round(recall, 3))
print("F1-score:", round(f1, 3))


dates = daily_df['date'].astype(str)[n_past:]

plt.figure(figsize=(18,6))

plt.plot(dates, real, label='Real Rainfall')
plt.plot(dates, pred, label='LSTM Prediction')
```

```
plt.xticks(
    np.arange(0, len(dates), 15),
    dates[::15],
    rotation=45
)

plt.xlabel("Date")
plt.ylabel("Rainfall (mm)")
plt.title("LSTM Rainfall Prediction")

plt.legend()
plt.grid()
plt.show()



plt.figure(figsize=(12,4))

plt.scatter(range(len(real_binary)), real_binary, label='Real (Anomaly)')
plt.scatter(range(len(pred_binary)), pred_binary, label='Pred (Anomaly)', alpha=0.6)

plt.title("Binary Anomaly Comparison (Threshold 기반)")
plt.legend()
plt.grid()
plt.show()
```